

Financial Forecasting by Multi-AI Agent System

Complete Phase-by-Phase Work Breakdown (Corrected Build Spec — India-Only)

B.Tech Capstone Project — step-by-step: every phase, every file, every function, every deliverable.

This is the **build document**: follow it task by task. It merges the detailed engineering steps with the fixes from the technical review, so nothing here leads to a dead end later.

Coverage: Indian equities only — NSE (.NS) primary, BSE (.BO) secondary. All prices in INR. All timestamps in Asia/Kolkata. Benchmark: NIFTY 50. All free data sources.
Golden rule: build the ruler before the thing you measure. Evaluation is Phase 2, not Phase 5.

Why India-only (defend this in the viva)

- It is the novelty.** The multi-agent debate architecture is already published (TradingAgents, arXiv:2412.20138) and that literature is overwhelmingly **US large-cap**. Indian equities are under-studied. Scope = contribution.
- It makes the evaluation honest.** One market means one trading calendar, one currency, one regulator, one filing deadline — the `as_of` protocol is provable rather than approximate. A worldwide claim would need per-market holiday calendars, FX conversion and per-regulator disclosure timing, none of which fit in a BTP.
- It buys India-specific engineering that actually earns marks:** point-in-time universe from NSE bhavcopy (kills survivorship bias), SEBI filing-date lag (kills fundamental leakage), NIFTY-relative scoring (kills the index-drift illusion). None of these exist in a "worldwide" build.
- The data is better here.** NSE publishes a free, keyless, complete daily archive back years — no other market in this project's budget gives that.

Anything not listed on NSE or BSE is out of scope. Not "supported later" — out of scope. If a ticker has no `.NS/.BO` resolution, the system rejects it at the boundary with a clear error.

Phase Map at a Glance

Phase	What you build	Result you can show
1	Foundation + data cache + single-LLM baseline	One command returns one real NSE forecast
2	Evaluation harness + leakage-free protocol	Any forecaster gets a scored results table
3	7 agents + debate + orchestrator + <code>--no-debate</code> switch	Type a ticker → full multi-agent forecast
4	Experiments & ablation	The tables and charts that earn marks
5	Dashboard + report + defence	Web app + thesis + slides

Golden rules that apply to every phase:

- Every data function takes an `as_of` **date**. No exceptions. (Prevents future-data leakage.)
- Every external call goes **through the cache**. (Prevents rate-limit death on demo day.)
- Every agent returns a **Pydantic model** with **citations**. (Prevents silent hallucination.)
- Every ticker is resolved to NSE/BSE at the boundary.** No market detection deeper in the stack.
- Commit to git after every working step.** `pip freeze > requirements.txt` after first good install.

Phase 1 — Foundation & Walking Skeleton

Goal: one command produces one real, schema-valid forecast — thin but complete, end to end.

Task 1.1 — Create the virtual environment

```
cd c:\Users\frien\OneDrive\Desktop\BTP
py -3.11 -m venv venv          # prefer 3.11; 3.13 is very new – see note
venv\Scripts\activate
python --version               # confirm it activated
```

Note on Python version: if `py -3.11` is unavailable, use `py -3` (3.13). If `pandas-ta-classic` or `langgraph` fails to install on 3.13, that is the signal to switch to a 3.11 venv. Decide by the install result, not by assumption.

Task 1.2 — Install dependencies (pinned)

```
pip install langgraph langchain-google-genai google-genai fastapi uvicorn \
            yfinance pandas pandas-ta-classic pydantic python-dotenv \
            requests feedparser httpx tenacity streamlit
pip freeze > requirements.txt      # DO THIS NOW, not at the end
```

Library	Purpose
langgraph	Multi-agent orchestration — the core framework
langchain-google-genai	Connect LangGraph nodes to Gemini
google-genai	Raw Gemini SDK (tighter control of structured output)
fastapi + uvicorn	Backend API server (Phase 5)
yfinance	NSE/BSE prices & financials — unofficial Yahoo scraper
pandas	Data manipulation
pandas-ta-classic	Technical indicators — maintained fork (imports as <code>pandas_ta_classic</code>)
pydantic	Structured agent output schemas
python-dotenv	Load API keys from <code>.env</code>
feedparser	Parse Indian financial RSS feeds (live demo only — see Task 2.5)
httpx	Async HTTP client — also fetches the NSE bhavcopy archive
tenacity	Exponential backoff/retry for 429s (new — not optional)
streamlit	Dashboard (primary UI, Phase 5)

(zipfile for the NSE bhavcopy is `stdlib` — nothing to install.)

Task 1.3 — Secure ALL API keys

- **Gemini** → aistudio.google.com — free tier.
- **FRED** → fredaccount.stlouisfed.org — free, separate registration. *Needed in Phase 2 for USD/INR and the India 10Y G-sec yield; get it now.*
- **GDELT needs NO key.** (NewsAPI is dropped — 1-month archive, 24h delay, localhost-only: useless for backtesting.)
- **NSE archives need NO key** — but they **do** need a browser `User-Agent` header (see Task 2.2).

Create .env in the project root:

```
GEMINI_API_KEY=your_key_here
FRED_API_KEY=your_key_here
```

Add .env to .gitignore **before your first commit** so keys never reach GitHub.

Task 1.4 — Project structure

```
BTP/
├── backend/
│   ├── __init__.py    ← REQUIRED (fixes "No module named 'backend'")
│   ├── config.py      ← settings + env loader + India constants
│   ├── cache.py       ← Task 2.1 (built first in Phase 2)
│   └── tools/         ← data tools (Phase 2)
│       ├── __init__.py
│       ├── tickers.py  ← NSE/BSE resolution + rejection (India-only boundary)
│       ├── calendar_nse.py ← NSE trading calendar (India-only)
│       ├── market_data.py
│       ├── indicators.py
│       ├── fundamentals.py
│       ├── news_sentiment.py
│       └── macro.py
│   ├── agents/        ← 7 agents (Phase 3)
│   │   └── __init__.py
│   ├── graph/         ← LangGraph orchestrator (Phase 3)
│   │   └── __init__.py
│   ├── eval/          ← evaluation harness (Phase 2) – NEW
│   │   └── __init__.py
│   └── api/           ← FastAPI (Phase 5)
│       └── __init__.py
├── frontend/         ← Streamlit/React (Phase 5)
├── tests/
│   └── test_env.py
├── data/             ← cache DB + frozen test set + bhavcopy cache live here
├── .env              ← never commit
├── .gitignore
├── requirements.txt
└── README.md
```

Every folder that is imported needs an `__init__.py`. This is the single most common Phase-1 bug.

Task 1.5 — config.py

```

import os
from dotenv import load_dotenv
load_dotenv()

class Config:
    GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
    FRED_API_KEY = os.getenv("FRED_API_KEY")
    # Flash-class model for specialists/debate (cheap, fast); reserve any Pro call for final
    synthesis
    GEMINI_MODEL = "gemini-flash-latest"
    CACHE_DB = "data/cache.db"

    # --- India-only constants ---
    MARKET_TZ = "Asia/Kolkata"
    CURRENCY = "INR"
    ALLOWED_SUFFIXES = (".NS", ".BO") # NSE primary, BSE secondary
    DEFAULT_SUFFIX = ".NS" # bare "RELIANCE" resolves to NSE
    BENCHMARK = "^NSEI" # NIFTY 50
    BENCHMARK_ALT = "^BSESN" # SENSEX
    VIX = "^INDIAVIX"
    USDINR = "INR=X"

```

Task 1.6 — tests/test_env.py (corrected)

```

import sys
print(f"Python: {sys.executable}")

for name, mod in [("yfinance", "yfinance"),
                  ("pandas_ta_classic", "pandas_ta_classic"), # NOT pandas_ta
                  ("google-genai", "google.genai"),
                  ("langgraph", "langgraph")]:
    try:
        __import__(mod)
        print(f"OK {name}")
    except ImportError as e:
        print(f"FAIL {name}: {e}")

# run as: python -m tests.test_env (from project root, so 'backend' is importable)
from backend.config import Config
print("OK config" if Config.GEMINI_API_KEY else "WARN GEMINI_API_KEY not set")

```

Run it: `python -m tests.test_env`

Task 1.7 — The single-LLM baseline (moved up from Phase 5 — critical)

Build `backend/baseline.py`: one Gemini call, given the same data, returns the **same** forecast schema as the full system.

This is the thing the multi-agent system must beat. It must exist before Phase 2 so the harness can score it.

Task 1.8 — One specialist end-to-end (the pattern)

Build the **Technical** agent fully (data → indicators → Gemini → `TechnicalReport`) as the template the other three will copy in Phase 3.

[OK] Phase 1 Exit Gate

`python -m forecast RELIANCE.NS --as-of 2025-01-15` returns a valid, schema-conformant forecast from **both** the baseline and the technical agent — **twice in a row**, the second run served from cache with **zero network calls**. A non-Indian ticker (`AAPL`) is **rejected with a clear error**, not silently fetched. `requirements.txt` committed. Git initialised.

Phase 2 — Data Layer & Leakage-Free Evaluation Harness

Goal: build every data tool *with time-travel built in*, then build the ruler that scores forecasts. **This is the academic core of the project.**

Task 2.1 — `cache.py` (BUILD FIRST — not optional)

- SQLite (in `data/cache.db`) sitting in front of **every** external call — prices, news, macro, and **LLM responses**.
- **Two cache classes** (this is the fix the old plan got wrong):

Class	Key	TTL
Historical / point-in-time	<code>(source, ticker, field, as_of)</code>	Permanent — the past never changes
Live quote (demo only)	<code>(source, ticker, field)</code>	15 min
LLM response	<code>sha256(prompt + model + nonce)</code>	Permanent

- Why permanent for history: a 15-min TTL forces constant re-fetching during ablation sweeps — triggering the exact 429s the cache exists to prevent.
- **Nonce on the LLM cache is mandatory for Phase 4 variance runs** — without it the repeat runs hit the cache and report zero variance, which is an artifact, not a finding.

Task 2.2 — `tickers.py` + `market_data.py` (India-only, time-aware)

`tickers.py` — the India boundary. Every entry point calls this first.

```
resolve(symbol) -> str          # "RELIANCE" -> "RELIANCE.NS"; "TCS.NS" -> "TCS.NS"
                                # "AAPL" -> raise UnsupportedMarketError
is_indian(symbol) -> bool
```

- Accept: bare symbol (defaults to `.NS`), explicit `.NS`, explicit `.BO`.
- Reject **everything else** — `AAPL`, `BP.L`, `7203.T`, `SAP.DE`, `0700.HK`, `BHP.AX` — with `UnsupportedMarketError("This system covers NSE/BSE-listed Indian equities only")`.
- Rejecting at the boundary is what makes "India-only" a property of the system rather than a habit. It is also a one-line test the examiner can run.

`market_data.py`

```
get_price_history(ticker, start, end, as_of)  # as_of caps the window; never returns rows after it
get_current_price(ticker)                    # live/demo path only
get_benchmark_history(start, end, as_of)      # ^NSEI — needed for NIFTY-relative scoring
```

- **Primary source: yfinance** with `.NS/.BO`. *Verified 2026-08-06*: `RELIANCE.NS` returns `currency: INR`, `exchange: NSE`, `timezone: Asia/Kolkata`, history from **1996**.
- **[!] The Stooq fallback in the old plan is dead.** *Verified 2026-08-06*: `stooq.com/q/d/1/` now returns a **JavaScript proof-of-work challenge page** to scripted clients, not CSV. Any `pd.read_csv` on that response yields an empty frame at best. **Do not build on it.**
- **Replacement fallback — the NSE bhavcopy archive (India-native, keyless, verified working):**

```
https://nsearchives.nseindia.com/content/cm/BhavCopy_NSE_CM_0_0_0_YYYYMMDD_F_0000.csv.zip
```

Verified 2026-08-06: HTTP 200, ~170 KB zip for 2025-01-15 — **requires a browser User-Agent header**; the default httpx UA gets blocked. Contains **every NSE cash-market instrument for that day**. Columns (UDiFF format): `TradDt`, `BizDt`, `Sgmt`, `Src`, `FinInstrmTp`, `FinInstrmId`, `ISIN`, `TckrSymb`, `SctySrs`, ..., `OpnPric`, `HghPric`, `LwPric`, `ClsPric`, `LastPric`, `PrvsClsPric`, ..., `TtlTradgVol`, `TtlTrfVal`, `TtlNbOfTxSExctd`, ... Filter to `SctySrs == "EQ"` and `FinInstrmTp == "STK"` for cash equities.

- **Normalise the bhavcopy to the yfinance schema** (`open/high/low/close/volume`) — a different schema is silent breakage exactly during the outage you built the fallback for.
- **Adjustment caveat — state it in the report:** bhavcopy prices are **unadjusted** for splits and bonuses; yfinance `auto_adjust` prices are adjusted. Mixing them across a corporate action produces a fake gap. Either adjust the bhavcopy yourself or record which source served each row and never mix sources within one series.
- Always fetch **≥ 2 years** of history regardless of the display window, so SMA-200 never returns NaN.

Task 2.3 — `calendar_nse.py` (India-only — new, and it prevents a whole class of bug)

```
is_trading_day(date) -> bool
previous_trading_day(date) -> date      # snap as_of backwards onto a real session
trading_days_between(start, end) -> int
add_trading_days(date, n) -> date      # the forecast horizon must be in SESSIONS, not calendar days
```

- **Derive the calendar from bhavcopy availability:** a file exists for a date NSE traded that day. No extra API, no hardcoded holiday list to go stale, and it is self-verifying.
- Cache the resolved calendar in `data/nse_calendar.json`; regenerate with one command.
- **Why this matters:** NSE has ~15 holidays a year and they do **not** match NYSE. An `as_of` of Diwali, or a 21-calendar-day horizon, silently corrupts every score in Phase 4. The horizon is **21 trading sessions**.

Task 2.4 — `indicators.py`

- `pandas_ta_classic` on the price DataFrame: **RSI(14), MACD(12/26/9), SMA 50 & 200, Bollinger, ATR**.
- Returns values + **plain-English reading** (e.g. RSI 72 → Overbought).
- Guard against short series (return `None` + a note, never a wrong number).
- **India-specific sanity rule:** NSE price bands cap most stocks at ±20% (±10%/±5%/±2% for many mid/small caps, and 5% for stocks in the F&O ban / surveillance list). **A single-session forecast outside the applicable band is arithmetically impossible** — clamp or flag it. Free the examiner a question and clamp it.

Task 2.5 — `fundamentals.py` (with the SEBI point-in-time rule)

```
get_fundamentals(ticker, as_of)      # P/E, EV/EBITDA, revenue growth, margins, D/E, market cap
get_earnings(ticker, as_of)         # last 4 quarters + surprise %
get_company_info(ticker)             # sector, industry, name, description
```

- **Explicit null handling:** yfinance coverage of `.NS` fundamentals is **patchier than for US tickers** — expect `None` for EV/EBITDA and some margin fields on mid- and small-caps. On a missing field the agent must say *"unavailable"* — **never invent a number**. Log the null rate per field; it is a table in the report.
- **SEBI filing-date lag (this replaces the vague "lag one quarter" rule):** under SEBI LODR Reg. 33, listed companies file quarterly results within **45 days** of quarter end (annual: **60 days**). So a quarter's numbers were **not public** until its filing date. Rule: **a quarter is visible only if `as_of` ≥ `quarter_end` + 45 days**. This is a concrete, citable, India-specific leakage fix — worth a paragraph in the report.
- **Documented limitation:** yfinance serves *today's* (restated) financials, not the originally reported figures. The 45-day rule fixes *timing* leakage, not *restatement* leakage. True point-in-time fundamentals are institution-priced; out of scope. Say so.

Task 2.6 — `news_sentiment.py` (GDELT, India-scoped, time-aware)

```
get_news(ticker, as_of)              # GDELT: filter to published_date <= as_of
score_sentiment(headlines)           # Gemini → polarity + relevance + confidence + catalysts
```

- **GDELT** replaces NewsAPI: 100+ languages, archive to 1979, free. Scope every query to India: `sourcecountry:india`, plus the company's **registered name** (query "Reliance Industries", not "Reliance" — the bare word is a mess of unrelated hits). Verify the filter syntax empirically in a probe before trusting it.

- **[!] GDELT rate-limits hard.** *Verified 2026-08-06:* back-to-back requests return "Please limit requests to one every 5 seconds" as the body, **HTTP 200** — it is **not** an exception and it **will** be cached as if it were data. **Detect that string explicitly and treat it as a retryable error**, then space requests ≥ 5 s via `tenacity`. Everything goes through the cache.
- **Dedup** by URL + title similarity before scoring — Indian outlets syndicate PTI/ANI copy heavily, so the same story appears 5–10 times and will skew polarity badly if counted once per copy.
- Indian RSS (Economic Times, Business Standard, Moneycontrol, Mint) via `feedparser` is **live-demo only** — RSS carries no usable archive, so it can never be used in the backtest. Keep the two paths in separate functions so this can't be violated by accident.
- GDELT gives metadata/tone, **not full article text** — adequate for sentiment; state as a limitation.
- **Prompt-injection guard:** headlines are untrusted web text. Delimit them clearly and instruct the model to treat them as data, not instructions. Validate output against the schema.

Task 2.7 — `macro.py` (Indian macro, time-aware)

```
get_macro_data(as_of)           # India rate/inflation/FX/oil regime, respecting data vintage
get_sector_performance(sector)  # NIFTY sectoral index relative performance
```

Verified FRED availability (keyless CSV probe, 2026-08-06 — check before building, these rot):

Indicator	FRED series ID	Freq	Last observation	Verdict
USD/INR	DEXINUS	Daily	2026-07-31	[OK] Use it
India 10Y G-sec yield	INDIRLTLT01STM	Monthly	2026-05	[OK] Use it
India call-money / short rate	IRSTCI01INM156N	Monthly	2026-05	[OK] Use it (policy-stance proxy)
India CPI	INDCPIALLMINMEI	Monthly	2025-03	[!] Stale — do not rely on
India IIP	INDPROINDMISMEI	Monthly	2023-01	[X] Dead
India discount rate	INTDSRINM193N	Monthly	2022-07	[X] Dead

- **Consequence:** CPI and the RBI **repo rate** are *not* reliably available on FRED. Get them from **RBI DBIE / MoSPI press releases**, or — the honest BTP answer — **hardcode the RBI repo-rate decision calendar** (it changes ~6 times a year, is public, and is trivially auditable) and **document CPI as a limitation**. Do not fabricate a series.
- **Data vintage still applies:** CPI for month M is released in month $M+1$. A macro reading must use only what was **published** on or before `as_of`, not what the series says about that month today.
- **India-specific drivers the macro agent must see** (all free via `yfinance`, verified 2026-08-06):

Driver	Symbol	Why it matters in India
NIFTY 50	<code>^NSEI</code>	Benchmark & regime
SENSEX	<code>^BSESN</code>	Cross-check
India VIX	<code>^INDIAVIX</code>	Regime / risk appetite
USD/INR	<code>INR=X</code> (or <code>DEXINUS</code>)	Importer cost, FII flows
Brent crude	<code>BZ=F</code>	India imports ~85% of its crude — a first-order macro driver
NIFTY IT	<code>^CNXIT</code>	Sector index example; IT is USD-revenue → inverse INR sensitivity

- `get_sector_performance` uses **NIFTY sectoral indices** (IT, Bank, Auto, Pharma, FMCG, Metal, Energy, Realty) — **not** US sector ETFs.

- Returns a **Tailwind / Neutral / Headwind** rating with reasoning.

Task 2.8 — The evaluation harness (backend/eval/) — the part the old plan lacked

1. **Determine the model's training cutoff empirically.** Run a **recall probe**: ask Gemini about firm-date Indian market outcomes it should not know (e.g. a specific NIFTY close, a specific quarterly result); verify it cannot recall them. Record the result — it is a figure in your report.
2. **Freeze the test set** — tickers, as-of dates, horizon (**21 trading sessions**, per Task 2.3) — in `data/testset.json`, **committed to git**. Never edit it after seeing results.
3. **Build the universe point-in-time, not from today's list.** Take the NIFTY 50 / NIFTY 100 constituents **as they stood on each as_of**, and confirm each ticker actually traded that day from the **bhavcopy for that date**. Using today's index membership is textbook **survivorship bias** — and because the bhavcopy is a complete daily snapshot, you are one of the few BTPs that can *prove* the universe was correct. Say that in the report.
4. **Test window must sit AFTER the training cutoff** (the recall probe confirms where that is). A within-training-window backtest measures memory, not forecasting — it is invalid.
5. **Metrics:**
 - Directional accuracy (up/down) — headline number
 - **NIFTY-relative direction** — did the stock beat the index? **Report this alongside raw direction.** Indian indices have trended up strongly; a model that says "Buy" every time can look like a 60%-accurate forecaster on raw direction. Excess-return scoring is what kills that illusion, and an examiner *will* ask.
 - MAE / MAPE on the price target (in ₹)
 - **Confidence calibration** (Brier score / reliability curve) — where multi-agent should win
 - **Cost per forecast** (tokens, seconds, calls) — to answer "is the debate worth it?"
 - A **naive baseline** (momentum persistence) **and a buy-and-hold-NIFTY baseline** to prove the LLM beats both a coin flip and the index
6. Harness runs *any* forecaster over the frozen set → CSV + markdown table.
7. **Record baseline results now** — these go in the report regardless of what happens later.

[OK] Phase 2 Exit Gate

`python -m evaluate --system baseline` produces a complete scored table over the frozen set, reproducibly. The recall-probe result is documented. Every tool works from the terminal, returns real **NSE/BSE** data in INR, is cached, respects `as_of`, and uses the **NSE trading calendar**.
`tickers.resolve("AAPL")` raises.

Phase 3 — Multi-Agent System

Goal: the full architecture, built against a harness that already exists.

Task 3.1 — Pydantic schemas (backend/agents/schemas.py)

- **TechnicalReport**: score (−10..+10), trend, support/resistance, indicator values, **citations**
- **FundamentalReport**: valuation score, intrinsic value, health status, **citations**
- **SentimentReport**: polarity (−1..+1), relevance, confidence, catalysts, **article links**
- **MacroReport**: regime rating, key indicators, reasoning
- **DebateArgument**: position (bull/bear), points, target, confidence
- **FinalForecast**: signal (Buy/Hold/Sell), price target range (₹), confidence %, risk report, **citations**

Gemini structured-output gotchas (verified) — test these in Phase 1, not here:

- Gemini **rejects** schemas using `Field(default=...)`. Use `Optional[...]` and post-process.
- Deeply nested models can be misread as tool calls (`Unknown tool name`). Keep schemas **flat**, prefer prefixed fields over deep object trees.
- Set `propertyOrdering` explicitly (up to ~15% quality gain on complex schemas).
- Count parse failures as a **reported metric**, not a silent retry.

India number-format trap: Indian sources write **1,00,000** (lakh) and **1,00,00,000** (crore), and copy says "₹2,500 crore". An LLM will happily parse these into the wrong magnitude. ****Normalise every monetary value to plain INR units before it reaches a schema field****, and make the reconciliation gate (Task 3.3) check magnitude, not just presence. A 100× error here is invisible until someone reads the target price.

Task 3.2 — The 4 specialist agents (backend/agents/)

Agent	System-prompt role	Tools (Phase 2)	Output
Technical	"Senior technical analyst, Indian equities — price action, momentum, patterns."	<code>market_data</code> , <code>indicators</code>	TechnicalReport
Fundamental	"Fundamental equity analyst, Indian listed companies — financial health & valuation."	<code>fundamentals</code>	FundamentalReport
Sentiment	"Indian market sentiment analyst — mood from domestic financial press."	<code>news_sentiment</code>	SentimentReport
Macro	"Indian macroeconomist — RBI policy, CPI, USD/INR, crude, FII/DII flows."	<code>macro</code>	MacroReport

Each agent receives the **as_of date** and must only use data at or before it. Every prompt states the market context explicitly: **NSE-listed, INR, Asia/Kolkata**. Do not let the model default to US assumptions (it will otherwise reason about the Fed and quote dollars).

Task 3.3 — Reconciliation gate (new — the anti-hallucination layer)

Before any agent's numbers propagate, **re-verify each number against the cached source data** (Bloomberg's ASKB pattern). A mismatch raises a warning, not a silent pass. Prove it works by injecting a fake number in a test and confirming the gate catches it.

- **Add an India magnitude check:** flag any value that is 100× / 10,000,000× off the source — that is the lakh/crore parse failure, and it is the most likely numeric error in this project.

Task 3.4 — Bull vs Bear debate engine

- **Bull:** receives all 4 reports → strongest upside case, growth catalysts, upside target.
- **Bear:** receives all 4 reports → downside risks, overvaluation signs, downside floor.
- **Bounded loop:** Bull → Bear rebuts → Bull counters → Bear final. **N=2 rounds** (unbounded debate burns quota fast). Each round logged for the Phase 5 dashboard.

Task 3.5 — Risk Officer / Synthesiser

- Receives all 4 reports + full debate transcript.
- Weighs evidence, applies risk rules → `FinalForecast` (signal, target range, confidence %, risk report).
- **Applies the NSE price-band sanity check** from Task 2.4 before emitting a target.
- Run at **temperature ≈ 0** for reproducibility.

Task 3.6 — LangGraph orchestrator (backend/graph/)

```
from typing import TypedDict, Optional, Annotated
import operator

class AgentState(TypedDict):
    ticker: str          # always NSE/BSE-resolved before entering the graph
    as_of: str
    # DISTINCT keys per specialist — avoids INVALID_CONCURRENT_GRAPH_UPDATE
    technical_report: Optional[TechnicalReport]
    fundamental_report: Optional[FundamentalReport]
    sentiment_report: Optional[SentimentReport]
    macro_report: Optional[MacroReport]
    debate: Annotated[list, operator.add]    # reducer for parallel/accumulated writes
    final: Optional[FinalForecast]
```

- **Parallel** fan-out: Technical, Fundamental, Sentiment, Macro run simultaneously.
- **Sequential** after: specialists → reconciliation → debate → risk officer → output.
- Add the **SQLite checkpoint** so a crash mid-run resumes, not restarts.
- **Wire the --no-debate switch from the start** — routes specialists straight to the synthesiser. Without it, the central research question cannot be answered.

[OK] Phase 3 Exit Gate

`python main.py TCS.NS --as-of 2025-06-02` runs all agents → prints the full forecast with debate.

(2025-06-01 is a Sunday — the calendar module should snap it; prove that too.)

The same system scores end-to-end on the Phase 2 harness with no manual steps, and the reconciliation gate catches a deliberately injected fake number **and a crore-magnitude error**.

Phase 4 — Experiments & Ablation

Goal: answer the research question with evidence. **This phase produces the marks.**

- **Primary comparison:** full multi-agent vs single-LLM baseline vs naive baseline vs buy-and-hold NIFTY.
- **Debate ablation:** full vs `--no-debate` — report accuracy delta **and** cost delta side by side.
- **Specialist ablation:** leave-one-out (drop each specialist). Expect at least one to add nothing — that is a finding, not a failure.
- **Calibration analysis:** reliability curves per configuration.
- **India-specific breakdowns (cheap to produce, and they read as real research):**
 - **Large-cap vs mid-cap** — fundamental data is sparser for mid-caps; does accuracy fall with the null rate? Ties directly back to the Task 2.5 null-rate table.
 - **By NIFTY sector** — IT (USD-revenue) vs Banks (rate-sensitive) vs FMCG (defensive).
 - **High vs low India-VIX regimes.**
- **Repeat runs** (≥ 3 , fixed temperature, **unique cache nonce per run**) to report variance — a single LLM run is an anecdote, and a cached repeat is a fake zero-variance result.
- **Qualitative failure analysis:** 5–10 wrong forecasts, diagnosed. Examiners reward this heavily.

[OK] Phase 4 Exit Gate

Every claim destined for the report is backed by a committed results file, regenerable by one command.

Phase 5 — Dashboard, Report & Defence

Goal: make it demonstrable and defensible. Deliberately last — high demo value, low research value.

Task 5.1 — FastAPI backend

```
POST /api/forecast/{ticker}    → runs pipeline, returns JSON
WS  /api/stream/{ticker}      → streams live agent progress ← owns the live UX flow
GET  /api/price/{ticker}      → chart data
```

- **Decide ownership now:** the WebSocket drives the live agent log (a 30–90 s POST times out on proxies).
- **Reject non-Indian tickers at the API boundary** with HTTP 400 and the same error message the CLI uses.
- **Security:** per-IP rate limit; explicit CORS (never `allow_origins=["*"]`); `.env` never committed; no secret behind a `NEXT_PUBLIC_` var; `pip audit` before submission.

Task 5.2 — Dashboard (Streamlit first)

NSE ticker search · price chart (**TradingView Lightweight Charts** — Recharts has no native candlestick) · **live agent execution log** · **Bull vs Bear split view** · **final forecast card** with confidence

- citations. Format money as ₹ with Indian grouping (lakh/crore) — it is a one-function detail that makes the demo look native. Move to React/Next.js **only if time genuinely remains** — a polished UI earns few marks against a missing ablation study.

Task 5.3 — BTP report

1. Introduction & problem statement
2. **Literature survey — position against TradingAgents (arXiv:2412.20138)** and cite the lookahead-bias work (arXiv:2512.23847). **Novelty = leakage-free measurement + ablation + Indian-market focus**, with the point-in-time universe and SEBI filing-lag rules as the concrete contributions.
3. System design (architecture, agent graph, sequence diagrams)
4. Implementation (stack, key algorithms, reconciliation, `as_of` design, NSE calendar)
5. **Results & evaluation** (leakage-free protocol, NIFTY-relative scoring, all ablations, calibration, cost, failure analysis)
6. Conclusion, limitations (restatement leakage, CPI/repo availability, GDELT text, unadjusted bhavcopy prices), future work
7. **Scope statement:** India-only is a deliberate design decision, with the four reasons at the top of this document. State it as a choice, never as a shortcut.

Task 5.4 — Slides + demo

15–20 slides: Problem → Solution → Architecture → Demo → Results → Conclusion. **Cache a known-good run** — never let a live API call decide whether your defence succeeds.

[OK] Phase 5 Exit Gate

A clean clone of the repo runs the demo, and every number in the report traces to a committed results file.

What Changed From the Old Breakdown (and why)

Change	Reason
Evaluation moved Phase 5 → Phase 2	Discovering a broken protocol in the last phase leaves no time to fix it
Single-LLM baseline moved into Phase 1	The core claim is comparative; the target must exist first

Change	Reason
<code>as_of</code> on every data tool	Without it the backtest leaks the future
Post-cutoff test window + recall probe	Within-training-window backtesting measures memory, not forecasting
NewsAPI → GDELT	NewsAPI free tier can't do historical; GDELT can, free
FRED key added to Phase 1	It was needed in Phase 2 but never provisioned
Two-class cache (permanent history)	A 15-min TTL re-fetches constantly and self-inflicts 429s
<code>--no-debate</code> ablation switch	The only way to measure if the debate earns its cost
Reconciliation gate + citations	Verified industry practice against numeric hallucination
Distinct state keys / reducer	Parallel LangGraph writes crash without this
Gemini schema gotchas surfaced	<code>Field(default=...)</code> rejected; nested schemas misread as tools
<code>pandas_ta_classic</code> import + <code>__init__.py</code>	The old test file fails forever without these
Security section added	Prompt injection, auth, key leaks — asked in any AI-agent viva
Dashboard demoted to Streamlit , last	High effort, low marks vs the evaluation work

India-only revision (this version)

Change	Reason
Worldwide → NSE/BSE only	Scope is the novelty; the literature is US-large-cap. One market = one calendar, one currency, one regulator — a <i>provable as_of</i> protocol
<code>tickers.py</code> hard boundary	"India-only" must be enforced by code, not by habit. <code>resolve("AAPL")</code> raises
Stooq fallback removed	<i>Verified 2026-08-06</i> : Stooq now serves a JS proof-of-work page to scripted clients — the fallback was already dead
NSE bhavcopy fallback added	Keyless, verified 200 OK, complete daily NSE snapshot — strictly better than Stooq ever was here
<code>calendar_nse.py</code> added	NSE holidays ≠ NYSE holidays; a 21- <i>calendar</i> -day horizon silently corrupts every Phase 4 score
Horizon = 21 trading sessions	Same reason, stated as a rule
Point-in-time universe from bhavcopy	Today's NIFTY 50 list is survivorship bias; the daily archive makes the correct universe <i>provable</i>
SEBI LODR 45-day filing rule	Replaces the vague "lag one quarter" with a citable, India-specific leakage fix
NIFTY-relative scoring added	Indian index drift makes raw directional accuracy flattering; an examiner will ask
FRED India series verified, not assumed	<i>Verified 2026-08-06</i> : USD/INR and 10Y yield are live; CPI, IIP and the discount rate are stale or dead — this would have blocked Phase 2
GDELT rate-limit body handling	<i>Verified 2026-08-06</i> : the limit message returns as HTTP 200 body text and would be cached as if it were data
NSE price-band sanity check	A $\pm 20\%$ band makes some LLM targets arithmetically impossible
Lakh/crore normalisation + magnitude check	Indian number formatting is a live 100× hallucination risk

Change	Reason
RSS split from GDELT	RSS has no archive — it can never touch the backtest; separate functions prevent the accident

References

- TradingAgents: Multi-Agents LLM Financial Trading Framework — **arXiv:2412.20138**
- Detecting Lookahead Bias in LLM Forecasts — **arXiv:2512.23847**
- Mitigating Look-Ahead Bias in Financial Backtesting with LLMs — **arXiv:2605.24564**
- LangGraph `INVALID_CONCURRENT_GRAPH_UPDATE` — LangChain docs
- Gemini structured output — googleapis/python-genai #699; pydantic-ai #3483
- **NSE daily bhavcopy archive** — nsearchives.nseindia.com/content/cm/ (UDiFF CSV, keyless, browser UA required)
- **SEBI LODR Regulation 33** — quarterly results within 45 days, annual within 60 days
- **FRED India series** — `DEXINUS`, `INDIRLT01STM`, `IRSTCI01INM156N` (verified live 2026-08-06)
- **RBI Database on Indian Economy (DBIE)** — repo rate, CPI, G-sec yields
- GDELT Project — gdeltproject.org · yfinance NSE/BSE suffixes (`.NS` / `.BO`) — help.yahoo.com